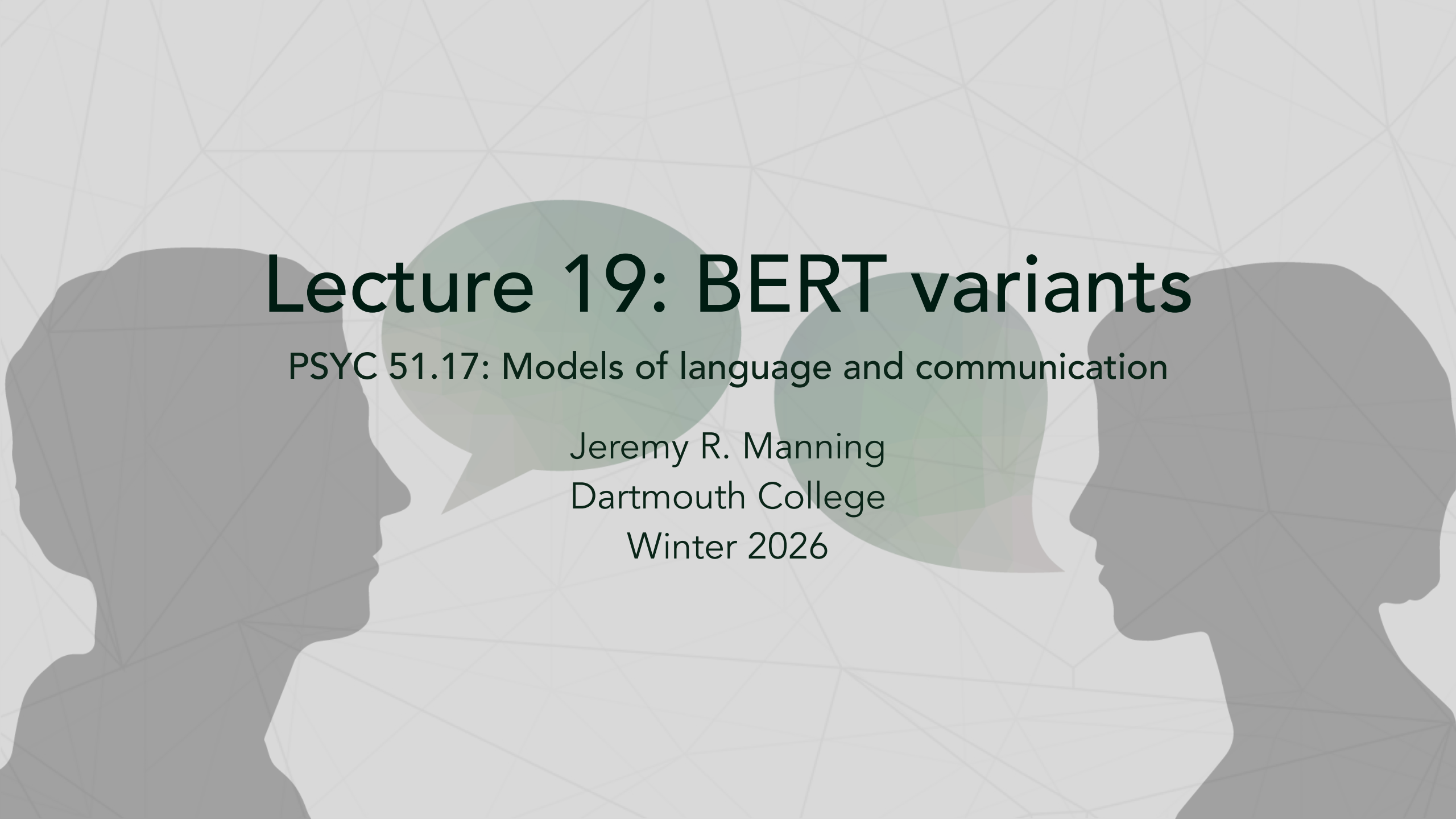




Lecture 19: BERT variants

PSYC 51.17: Models of language and communication

Jeremy R. Manning
Dartmouth College
Winter 2026

Learning objectives

By the end of this lecture, you will be able to...

1. Identify key limitations of the original BERT training procedure
2. Explain how RoBERTa, ALBERT, DistilBERT, ELECTRA, and DeBERTa each address different limitations
3. Compare parameter efficiency, training efficiency, and inference speed across variants
4. Select the appropriate variant for a given task and resource constraint
5. Argue whether encoder models remain relevant in the era of GPT-4

What could be improved?

BERT's key limitations (from Lecture 18)

Training procedure: NSP may hurt performance; static masking reuses the same masks every epoch; only 15% of tokens provide training signal.

Scale: Trained on only 3.3B words with 100K steps — modern datasets are 100× larger.

Efficiency: 110M parameters are all active for every input. Large memory footprint for deployment.

Companion notebook



Companion Notebook — try different BERT variants hands-on

RoBERTa: robustly optimized BERT

Key idea: better training = better performance

RoBERTa (Liu et al., 2019) keeps BERT's architecture but fixes the training recipe:

1. **Remove NSP** — Next Sentence Prediction hurt performance. Use only MLM with full sentences.
2. **Dynamic masking** — Generate a new masking pattern every time a sequence is seen, instead of reusing the same mask.
3. **Larger batches, more data** — Batch size 8K sequences (vs BERT's 256), 160GB text (vs 16GB), 500K steps (vs 100K).
4. **Longer sequences** — Train on longer contiguous text for better long-range understanding.

Takeaway

Training procedure matters as much as architecture. RoBERTa shows that BERT was significantly **undertrained**.

Dynamic masking

RoBERTa's key innovation

BERT used **static masking** — the same mask positions every epoch, risking memorization. RoBERTa generates **new masks on-the-fly** during training:

- Epoch 1: "My [MASK] is cute"
- Epoch 2: "My dog [MASK] cute"
- Epoch 3: "My dog is [MASK]"

More diverse training signal → better generalization. Combined with removing NSP and training longer on more data, this alone accounts for most of RoBERTa's gains.

RoBERTa results

Consistent improvements over BERT

Task	BERT-Large	RoBERTa	Improvement
SQuAD 2.0	83.1	89.4	+6.3
MNLI	86.7	90.2	+3.5
SST-2	94.9	96.4	+1.5
RACE	72.0	83.2	+11.2

Key findings

- Dynamic masking consistently outperforms static masking
- More data + longer training = substantial gains
- Removing NSP improves downstream task performance
- Some tasks see enormous gains (RACE: +11.2 points!)

ALBERT: a lite BERT

Key idea: parameter sharing for efficiency

ALBERT (Lan et al., 2019) dramatically reduces BERT's parameter count through two innovations:

1. Factorized embedding parameters:

- BERT: vocabulary (30K) $\times$ hidden size (768) = 23M parameters
- ALBERT: vocabulary (30K) $\times$ embedding size (128) + embedding (128) $\times$ hidden (768) = 3.9M parameters
- **83% fewer embedding parameters**

2. Cross-layer parameter sharing:

- All 12 transformer layers share the *same* weights
- Like running the same layer 12 times (iterative refinement)
- **89% fewer transformer parameters**

3. Sentence Order Prediction (SOP):

- Replaces NSP with a harder task: are sentences A, B in correct order or

ALBERT parameter efficiency

Dramatic parameter reduction

Model	Layers	Hidden size	Parameters
BERT-base	12	768	110M
ALBERT-base	12	768	12M
ALBERT-large	24	1024	18M
ALBERT-xlarge	24	2048	60M
ALBERT-xxlarge	12	4096	235M

Factorized embedding in code

```
1 import torch.nn as nn
2
3 # BERT: direct embedding (30K vocab × 768 hidden = 23M
  params)
4 bert_embed = nn.Embedding(30000, 768)
```


ALBERT parameter efficiency

Factorized embedding in code

9 # 83% fewer embedding parameters!

...continued

Cross-layer parameter sharing

BERT vs ALBERT layer structure

```
1 class BERT:
2     def __init__(self):
3         # Each layer has unique parameters
4         self.layers = [TransformerLayer() for _ in range(12)]
5         # 12 × 7M params = 85M params in transformer layers
6
7     def forward(self, x):
8         for layer in self.layers:
9             x = layer(x)    # Different weights each time
10        return x
11
12 class ALBERT:
13     def __init__(self):
14         # Single shared layer
15         self.shared_layer = TransformerLayer()
```

continued...

Cross-layer parameter sharing

BERT vs ALBERT layer structure

```
16         # 1 × 7M params = 7M params (89% reduction!)
17
18     def forward(self, x):
19         for _ in range(12):
20             x = self.shared_layer(x) # Same weights reused
21         return x
```

...continued

Trade-off

ALBERT has 89% fewer parameters but the **same compute cost** — it still performs 12 forward passes through the transformer layer. The savings are in memory, not speed.

DistilBERT: knowledge distillation

Key idea: train a small model to mimic a large model

Knowledge distillation (Sanh et al., 2019) compresses BERT into a smaller, faster model:

- **Teacher:** Full BERT-base (12 layers, frozen)
- **Student:** DistilBERT (6 layers, trainable)
- **Training signal:** Student learns to match the teacher's *soft probability distributions*, not just the hard labels

The teacher's "wrong" predictions contain useful information — e.g., predicting "cat" is more likely than "car" for a masked animal slot tells the student about semantic similarity.

Results

Metric	BERT-base	DistilBERT	Change
Parameters	110M	66M	40% smaller
Inference speed	1×	1.6×	60% faster
GLUE score	79.6	77.0	97% retained

DistilBERT training

Knowledge distillation training loop

```
1 teacher = BertModel.from_pretrained("bert-base") # 12 layers,
   frozen
2 student = DistilBertModel(num_layers=6)           # 6 layers,
   trainable
3
4 for batch in training_data:
5     # Teacher provides "soft targets" (probability
   distributions)
6     with torch.no_grad():
7         teacher_logits = teacher(batch) # e.g., [0.7, 0.2, 0.1,
   ... ]
8
9     # Student tries to match teacher's distribution
10    student_logits = student(batch)
11
12    # Distillation loss: KL divergence between soft
```

continued...

DistilBERT training

Knowledge distillation training loop

```
14     loss_distill = KL_divergence(  
15         softmax(student_logits / T),  
16         softmax(teacher_logits / T)  
17     )  
18     loss_mlm = masked_lm_loss(student_logits, labels)  
19  
20     loss = 0.5 * loss_distill + 0.5 * loss_mlm
```

...continued

ELECTRA: efficient learning from all tokens

Key idea: learn from 100% of tokens, not just 15%

ELECTRA (Clark et al., 2020) uses a **generator-discriminator** setup:

1. A small **generator** (like a mini-BERT) fills in masked tokens with plausible replacements
2. A **discriminator** classifies *every* token as original or replaced
3. The discriminator provides a training signal for **all tokens** — not just the 15% that were masked

ELECTRA training example

```
1 sentence = "The chef cooked a delicious meal"
2 masked   = "The chef [MASK] a delicious meal"
3
4 # Small generator fills in the mask
5 generator_output = generator(masked) # Predicts: "ate" (plausible
6 corrupted = "The chef ate a delicious meal"
```

ELECTRA: efficient learning from all tokens

ELECTRA training example

```
9 | discriminator(corrupted)
10 | # Output: [orig, orig, REPLACED, orig, orig, orig]
11 | # Loss computed on ALL 6 tokens (not just 1 masked token!)
```

...continued

ELECTRA efficiency

Learning from every token

ELECTRA learns from **100% of tokens** (every position gets a real/replaced label), compared to BERT's 15%. This $6.7\times$ increase in training signal means ELECTRA reaches BERT-level performance with **4× less compute**.

When to use ELECTRA

ELECTRA is ideal when you have limited compute budget:

- ELECTRA-Small outperforms BERT-Small
- ELECTRA-Base is competitive with BERT-Large
- With the same compute budget, ELECTRA consistently wins

Variant comparison summary

Choosing the right BERT variant

Model	Key innovation	Best for
BERT	MLM + NSP	Baseline, well-understood
RoBERTa	Better training recipe	Maximum quality
ALBERT	Parameter sharing	Memory-constrained deployment
DistilBERT	Knowledge distillation	Speed-critical production
ELECTRA	Replaced token detection	Limited training budget

General guidelines:

- **Best quality:** RoBERTa-Large
- **Best efficiency:** DistilBERT
- **Limited memory:** ALBERT
- **Limited training budget:** ELECTRA
- **Good default:** RoBERTa-Base or BERT-Base

Other notable BERT variants

The BERT family keeps growing

DeBERTa (Microsoft, 2020): Disentangled attention separates content and position representations. Enhanced mask decoder. State-of-the-art on SuperGLUE.

SpanBERT (Facebook, 2019): Masks random contiguous *spans* instead of individual tokens. Span boundary objective. Better for extractive tasks (QA, coreference).

ERNIE (Baidu, 2019): Entity-level and phrase-level masking. Knowledge-enhanced pre-training. Strong on Chinese NLP tasks.

BART (Facebook, 2019): Encoder-decoder architecture (not encoder-only). Denoising autoencoder with various corruption strategies. Excellent for generation tasks.

Model selection guide

How to choose the right model for your task

Step 1: What are your constraints?

- Need maximum quality? → **RoBERTa-Large**
- Need fast inference? → **DistilBERT**
- Need small memory footprint? → **ALBERT**
- Limited training compute? → **ELECTRA**
- Need text generation? → Consider **BART** or **GPT** instead

Step 2: Consider your domain

- Biomedical text? → **BioBERT**, **PubMedBERT**
- Scientific text? → **SciBERT**
- Legal text? → **LegalBERT**
- Multilingual? → **mBERT**, **XLM-RoBERTa**

Step 3: Start simple, iterate

- Begin with **bert-base-uncased** as a baseline

Using different variants with HuggingFace

Easy model switching with AutoModel

```
1  from transformers import AutoModel, AutoTokenizer
2
3  # All variants share the same API – just change the model name!
4
5  # BERT
6  model = AutoModel.from_pretrained("bert-base-uncased")
7  tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
8
9  # RoBERTa
10 model = AutoModel.from_pretrained("roberta-base")
11 tokenizer = AutoTokenizer.from_pretrained("roberta-base")
12
13 # ALBERT
```

continued...

Using different variants with HuggingFace

Easy model switching with AutoModel

```
14 model = AutoModel.from_pretrained("albert-base-v2")
15 tokenizer = AutoTokenizer.from_pretrained("albert-base-v2")
16
17 # DistilBERT
18 model = AutoModel.from_pretrained("distilbert-base-uncased")
19 tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
20
21 # ELECTRA
22 model = AutoModel.from_pretrained("google/electra-base-discriminator")
23 tokenizer = AutoTokenizer.from_pretrained("google/electra-base-discriminator")
```

...continued

Benchmarking variants

Comparing models on sentiment analysis

```
1 import time
2 from transformers import pipeline
3
4 models = {
5     "bert-base": "textattack/bert-base-uncased-SST-2",
6     "distilbert": "distilbert-base-uncased-finetuned-sst-2-english",
7     "albert": "textattack/albert-base-v2-SST-2",
8 }
9 test_texts = ["This movie was fantastic!", "I hated every minute."] * 100
10
11 for name, model_id in models.items():
12     pipe = pipeline("sentiment-analysis", model=model_id)
13     start = time.time()
14     results = pipe(test_texts)
```

Architecture evolution

From BERT to ModernBERT: 6 years of progress

Model	Year	Key innovation	Quality vs BERT
BERT	2018	MLM + NSP	Baseline
RoBERTa	2019	Better training recipe	+3–11 pts
ALBERT	2019	Parameter sharing	89% fewer params
DistilBERT	2019	Knowledge distillation	97% quality, 60% faster
ELECTRA	2020	Learn from all tokens	4× less compute
DeBERTa	2020	Disentangled attention	SOTA on SuperGLUE
ModernBERT	2024	Modern training + RoPE + Flash Attention	SOTA on GLUE, retrieval

ModernBERT deep dive

Warner et al. (Dec 2024): bringing modern LLM techniques to encoders

ModernBERT applies 6 years of decoder innovations to the encoder architecture:

Innovation	BERT (2018)	ModernBERT (2024)
Position encoding	Learned absolute (512 max)	RoPE (8,192 tokens)
Attention	Full quadratic	Flash Attention + alternating global/local
Padding	Processes pad tokens	Unpadding (only real tokens)
Training data	3.3B words	2 trillion tokens (600× more)
Code understanding	None	Trained on code corpora

Results

SOTA on GLUE, retrieval (MTEB), and code understanding benchmarks. Available as `answerdotai/ModernBERT-base` and `answerdotai/ModernBERT-large` on HuggingFace. Proves that encoder architecture still has room to grow when given modern training techniques.

Gemma Encoder and the encoder renaissance

Google bets on encoders again (2025)

Gemma Encoder (2025): Google's first encoder-only model since BERT, built by repurposing Gemma decoder weights for bidirectional encoding. Competitive with ModernBERT on sentence embedding tasks.

Why this matters

If Google — a company betting heavily on decoder-only models (Gemini) — still releases an encoder model, it signals that encoders serve a purpose decoders can't efficiently fill. The encoder isn't dead; it's being **modernized**.

Production deployment patterns

Getting encoders from prototype to production

Optimization	Speedup	Memory savings	Quality loss
ONNX Runtime	2–5×	Moderate	None
TensorRT (NVIDIA)	5–10×	Moderate	None
INT8 quantization	2–4×	4× smaller	<1%
Distillation (→ DistilBERT)	1.6×	40% smaller	~3%
Full pipeline (distill → quantize)	10–20×	8× smaller	~4%

The cost argument

DistilBERT + INT8 quantization handles classification at ~\$0.001/M tokens and ~2ms per request. GPT-4 costs ~\$30/M tokens at ~500ms per request. For high-volume single-task workloads, optimized encoders are **30,000× cheaper** and **250× faster**.

Is the encoder dead?

Discussion: GPT-4 can classify text via prompting. Do we still need encoders?

The case for "yes, encoders are obsolete":

- Decoder-only models (GPT-4, Claude) can do classification, NER, QA via prompting
- One model for all tasks vs. fine-tuning separate models
- In-context learning eliminates the need for task-specific architectures

The case for "no, encoders still matter":

- ModernBERT (2024) achieves SOTA on retrieval and classification — faster and cheaper than any decoder
- Gemma Encoder (2025) proves Google still sees value in the architecture
- Encoders are 10–100× cheaper to run than decoder models for classification tasks
- Most production search and retrieval systems still use encoders (Sentence-BERT, E5, NV-Embed)

The real answer: It depends on your constraints. Encoders win on cost and latency

Discussion

Questions to consider

1. **Training vs architecture:** RoBERTa shows that training matters enormously. How much of BERT's "limitations" were really just undertrained models?
2. **The distillation paradox:** Why does a student model learn *better* from soft probability distributions than from hard labels? What "dark knowledge" is in the teacher's mistakes?
3. **Encoders in 2026:** Google, Hugging Face, and others are *still* releasing encoder models. If decoders can do everything, why? What does this tell us about the efficiency-flexibility tradeoff?
4. **Parameter sharing (ALBERT):** All 12 layers share the same weights and it still works. What does this imply about what transformer layers actually learn?

Further reading

Further reading

Liu et al. (2019, *arXiv*) "RoBERTa: A Robustly Optimized BERT Pretraining Approach" — Better training recipe, same architecture.

Lan et al. (2019, *ICLR*) "ALBERT: A Lite BERT" — 89% parameter reduction via sharing.

Sanh et al. (2019, *NeurIPS Workshop*) "DistilBERT" — Knowledge distillation for 60% speedup.

Clark et al. (2020, *ICLR*) "ELECTRA" — Learn from all tokens, not just masked ones.

He et al. (2020, *ICLR*) "DeBERTa" — Disentangled attention, SOTA on SuperGLUE.

Warner et al. (2024, *arXiv*) "ModernBERT" — Modern encoder with RoPE + Flash Attention.

Google (2025, *arXiv*) "Gemma Encoder" — Encoder-only Gemma for sentence embeddings.

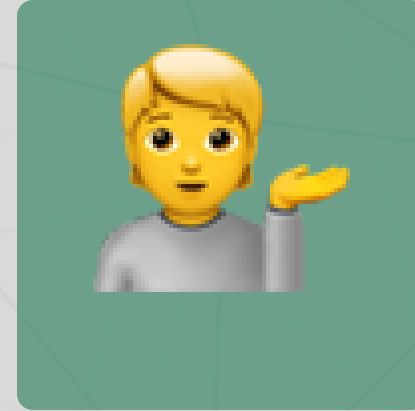
Questions?



Email



Discord



Office hours

Up next...

Applications of encoder models: industry, neuroscience, and the "understanding" debate